

כשתיארונו את האלגוריתמים השונים במהלך הקורס עד כה, הנחנו שהשלבים השונים בהם מבוצעים בזה אחר זה במעבד של המחשב בו הם מיושמים. למעשה, מעבדי המחשב מבצעים פעולות שונות בו זמנית וכך מאיצים את החישוב. לדוגמה, כאשר המעבד מבצע פעולה אריתמטית על גדלים מסוימים, חלקים אחרים במעבד מכינים בו זמנית את הגדלים הנדרשים לחישובים הבאים על ידי קריאתם מהזיכרון. ההנחיות לביצוע הפעולות במעבד, נקבעות על ידי המפרשן (interpreter) או המהדר (compiler) המתרגמים את תוכנת המחשב שנכתבה על ידי המשתמש. מקבול שכזה של החישובים נעשה כבר עשרות שנים והוא כמעט¹ "שקוף" למשתמש, מכיוון שהוא נעשה אוטומטית.

מהלך השנים, הייתה כידוע התקדמות רבה בכוח החישוב. התקדמות זו התבססה על היכולת הטכנולוגית למזער יותר ויותר את רכיבי המוליכים למחצה. כך, ניתן היה להאיץ את קצב² ביצוע הפעולות וכן להגדיל את כמות הטרנזיסטורים במעבדים. הגדלה זו, בשילוב עם שיפורים מתמידים במהדרים, בתוכנות ההפעלה של המעבדים וברכיבי השירות של המעבדים (כגון רכיבי ניהול זיכרון) אפשרו לטייב את ביצוע המקבול האוטומטי ולשפר את מהירות הביצוע של אלגוריתמים נומריים באופן מדהים. בנוסף, אפשרה העצמת כוח המחשוב לפתור נומרית בעיות עם מספר גדל והולך של משתנים, לדוגמה העלאת הרזולוציה המרחבית בסימולציות המשמשות לתחזיות מזג אוויר.

עם הזמן, התפתחו מספר טכנולוגיות שונות שהתבטאו בגידול היכולת לבצע פעולות שונות במקביל בתוך המעבדים. אין זה המקום לסקור טכנולוגיות אלו, אך נזכיר רק חלק מהמונחים המתארים אותן: מעבד סופרסקלרי, מעבד ווקטורי, יכולת הצנרה (pipeline). כאמור לעיל, מכנה משותף של יכולות אלו הוא שהן מתבצעות אוטומטית מבחינת המשתמש ולכן הוא לא חייב להכירן כדי ליהנות מהתועלת שבהן. מכנה משותף נוסף ליכולות אלו היא הסיבוכיות ההולך וגובר של מבנה המעבד הכרוך ביישומם ובאבולוציה שלהם ושל המהדרים הנדרשים לנצל את תכונותיהם. דרך חלופית להעצמת כוח המחשוב היא פשוט להכפיל את כמות המעבדים. הכפלה זו מיושמת גם על ידי ייצור מעבדים מרובי ליבות חישוב (cores), שהם למעשה מעבדים עצמאיים ה"מודפסים" על אותה פיסת סיליקון וגם על ידי שילוב מספר מעבדים על לוח האם של המחשב. פתרונות אלו קיימים כיום בכל רמה של מחשוב, מהטלפונים הסלולריים והמחשבים הניידים האישיים ועד מחשבי העל החזקים ביותר (בהם מחברים אלפי יחידות עיבוד³ על ידי תקשורת מהירה).

במצב בו כוח המחשוב מנוצל לביצוע תהליכים שונים, כגון בטלפון סלולרי בו בדרך כלל מתבצעים בו זמנית תהליכים רבים של מערכת ההפעלה ושל המשתמש, קל לנצל את ריבוי ליבות החישוב על ידי הקצאה של תהליכים שונים לליבות החישוב הפנויות. לעומת זאת, כאשר יש צורך בשימוש בכל הליבות כדי להאיץ חישוב יחיד, נדרשת התאמה של האלגוריתם בו משתמשים, והדבר כמעט תמיד לא נעשה אוטומטית. מכיוון

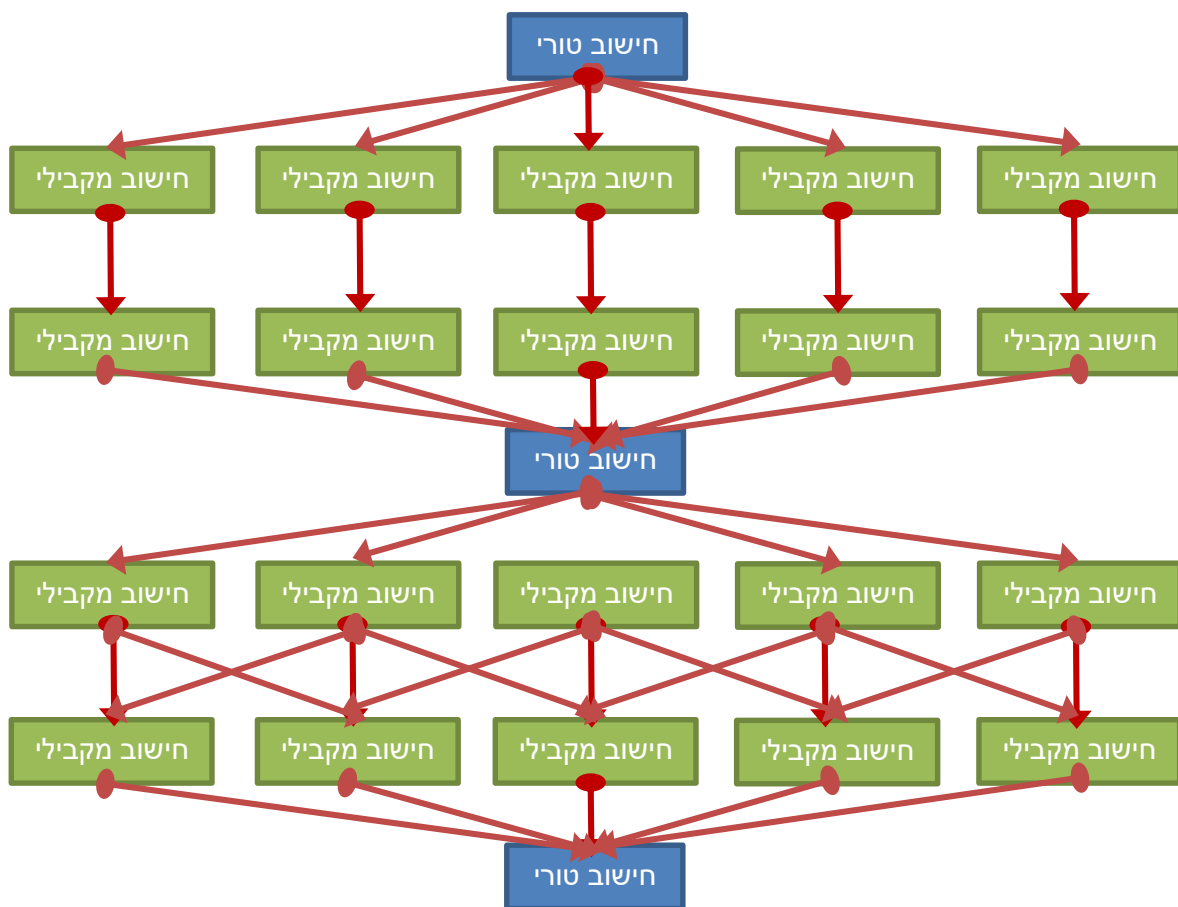
¹ מבנה תוכנית המחשב עשוי להשפיע על היעילות שלה, לכן כאשר יש חשיבות גדולה להאצת זמן החישוב, המשתמש יכול לנצל ידע על המהדר/מפרשן ועל החומרה כדי להתאים ולייעל את תוכנית המחשב וזאת למרות שמקבול שכזה נעשה אוטומטית,

² האצת קצב ביצוע הפעולות על ידי הגדלת תדר השעון (clock rate) של המעבד הקובע את יחידת הזמן הבסיסית בביצוע פעולות על ידי המעבד

³ כל יחידת עיבוד שקולה למחשב בו יש לוח אם עם מספר מעבדים ורכיבים נוספים.

שכך, נסביר על קצה המזלג, בסיכום זה ובסיכום הבא, על דרכי מקבול אפשריות ומהם השיקולים המרכזיים בהם צריך להתחשב בתהליך המקבול. ראוי לציין שכיום השימוש ביכולת חישוב מקבילי הופך להיות סטנדרטי בכל מקום בו החישובים דורשים כוח חישובי גדול. זאת, למרות הצורך בשינוי והתאמת האלגוריתמים לשימוש במחשוב מקבילי.

מקבול של אלגוריתם תלוי כמובן באלגוריתם עצמו. במצב בו החישוב הכולל מורכב מחישובים רבים שאינם תלויים זה בזה, המקבול הוא לרוב פשוט. מאידך, כאשר מבצעים חישובים רבים שכל אחד מהם תלוי בתוצאה של קודמו הרי שמקבול אינו אפשרי (או לא יעיל). בציור הבא ננסה להמחיש באופן סכמטי מושגים חשובים במקבול על ידי תיאור ציורי של דוגמה לאלגוריתם לאחר מקבול:



תרשים זה מתאר אלגוריתם בו יש שבעה שלבי חישוב שונים, חלקם טוריים וחלקם מקביליים. כמו כן מתאר התרשים את זרימת המידע בין השלבים השונים. השלב הראשון הוא טורי וכולל מן הסתם את הגדרת הבעיה המחושבת וקריאת נתוני הקלט הרלוונטיים. עם סיומו, מתחיל שלב 2 בו מבוצעים חמישה חישובים במקביל בו כל חישוב תלוי בנתונים שהתקבלו משלב 1. גם החישובים בשלב 3 נעשים במקביל כשכל חישוב תלוי בתוצאת חישוב מקבילי קודם שנעשה בשלב 2 ולכן לא ניתן לבצע את החישובים של שלב 2 ו-3 במקביל בו זמנית (גם אם יש חומרה המסוגלת לבצע עשרה חישובים במקביל). שלב 4 הוא טורי והוא משתמש במידע שחושב על ידי כלל החישובים המקביליים שבוצעו בשלב 3. שלב 5 מבוצע במקביל כשכל חישוב תלוי בתוצאות שחושבו בשלב 4 הטורי. גם שלב 6 הוא מקבילי אלא שבו כל חישוב תלוי בתוצאות

שחושבו במספר חישובים שונים של שלב 5. השלב האחרון הוא טורי ומסתמך על תוצאות מכלל החישובים המקביליים של שלב 6 וכולל מן הסתם את הפלט הסופי של החישוב.

דוגמה זו ממחישה את מה שנדרש לעשות לצורך המקבול. יש צורך להפריד את האלגוריתם לשלבים טוריים ולשלבים מקביליים בהם אין תלות הדדית ויש צורך בניתוח זרימת המידע (או התלות בין משתני החישוב) בין השלבים השונים.

מכיוון שהסיבה המרכזית למקבול היא הרצון להאיץ את ביצוע החישוב, חשוב לנתח את פקטור ההאצה המתקבל. נניח שיש בעיה חישובית שגודלה ניתן לאפיון על ידי פרמטר n (לדוגמה – כמות נקודות הדגימה בחישוב מונטה-קרלו או מספר תאי החישוב בסימולציה הידרודינמית) ונניח שהיא מוקבלה ל- c ליבות. נגדיר את פקטור ההאצה כיחס בין זמן הריצה הסדרתית לזמן הריצה המקבילית. באופן אידאלי פקטור ההאצה יהיה c , אך כפי שהודגם בתרשים לעיל, חלק מהחישוב נשאר טורי (או סדרתי) ויש תקורה הנובעת מהתקשורת בין החישובים הנדרשת לצורך המקבול ומחוסר יעילות נוספת. נגדיר את הגדלים הבאים:

$s(n)$ - זמן הריצה של החלק הטורי על ליבה אחת (תלוי כמובן ב- n).

$p(n)$ - זמן הריצה של החלק המקבילי על ליבה אחת (תלוי כמובן ב- n).

$o(n, c)$ - זמן ה"מתבזבז" על ידי התקורה מהתקשורת בין החישובים (תלוי ב- n וב- c).

$l_b(n, c)$ - פקטור איזון העומסים ו(חוסר)יעילות המקבול ($1 \leq$). מוגדר כך שזמן הריצה של החלק

המקבילי על c ליבות יהיה $\frac{p(n)}{c} l_b(n, c)$.

זמן הריצה הסדרתית הוא כמובן $s(n) + p(n)$.

זמן הריצה המקבילית הוא $s(n) + l_b(n, c) \frac{p(n)}{c} + o(n, c)$.

מכאן נקבל שההאצה היא: $speedup(n, c) = \frac{s(n)+p(n)}{s(n)+l_b(n,c)\frac{p(n)}{c}+o(n,c)}$

ניתן לראות שאכן במקבול אידאלי עבורו: $s = 0, l_b = 1, o = 0$ נקבל: $speedup = c$

כמו כן קל לנתח מקרה בו החלק הטורי מהווה שבר f מזמן החישוב הסדרתי ואין תקורה וחוסר יעילות נוספים. כלומר, $f = \frac{s}{s+p}, l_b = 1, o = 0$. נקבל: $speedup = \frac{1}{f + \frac{1-f}{c}}$. נוסחה זו נקראת

Amdahl's Law וכדי להבין את משמעותה נסתכל במצב בו מספר הליבות אינו מוגבל כך ש- $\frac{1-f}{c}$ קטן

כרצוננו. במקרה זה נקבל $speedup = \frac{1}{f}$. משמעות הדבר לדוגמה היא שאם 5% מזמן הריצה הוא טורי

הרי שגם אם יהיו לנו מיליון ליבות והמקבול של החלק המקבילי הוא אידאלי וללא שום תקורות, לא נוכל לקבל האצה של יותר מפקטור 20. בפועל, מכיוון שיש תקורות וחוסר יעילות והם בדרך כלל מתגברים עם עלייה במספר הליבות הרי שנקבל אפילו ירידה בהאצה כשנעלה את מספר הליבות יותר מידי.

הסיבה לפגיעה בהאצה לפי חוק Amdahl היא שזמן הריצה של החלק המקבילי יורד לאפס כשמעלים את כמות הליבות. מצב זה מתייחס למקרים בהם גודל הבעיה קבוע, אך במקרים רבים המעבר לחישוב מקבילי מנוצל לשיפור החישוב על ידי הגדלת הפרמטר n , לדוגמה, שיפור הרזולוציה המרחבית בחיזוי מזג אוויר.

מקרים שכאלה מאופיינים בכך שזמן הריצה המקבילי $\frac{p(n)}{c}$ הוא קבוע עם הגדלת מספר הליבות. ננתח מקרה שכזה בו בנוסף זמן הריצה של החלק הטורי s לא תלוי בגודל הבעיה. במקרה זה הזמן היחסי של החלק הטורי מתוך זמן הריצה הכולל על c ליבות יהיה קבוע: $g = \frac{s}{s+\frac{p}{c}}$ ונניח לשם הפשטות שהמקבול הוא אידאלי $l_b = 1, o = 0$. במקרה זה נקבל⁴ שההאצה היא: $speedup = \frac{s+p}{s+\frac{p}{c}} = c - g(c-1)$. כלומר, תלות ההאצה היא ליניארית ב- g וליניארית ב- c וניתן לקבל האצה משמעותית גם כשעולים הרבה בכמות הליבות. נוסחה זו נקראת Gustafson-Barsis's Law. בדיקת הסילומיות (scalability) בגודל בעיה קבוע (חוק Amdahl במקרה אידאלי) נקראת סילום חזק (strong scaling), ובדיקת סילומיות כשגודל הבעיה גדל (חוק Gustafson במקרה האידאלי) נקראת סילום חלש (weak scaling).

דוגמה למקבול של חישובי מונטה-קרלו

חישובי מונטה-קרלו רבים ניתנים לתיאור על ידי הקוד המדומה (pseudo code) הבא:

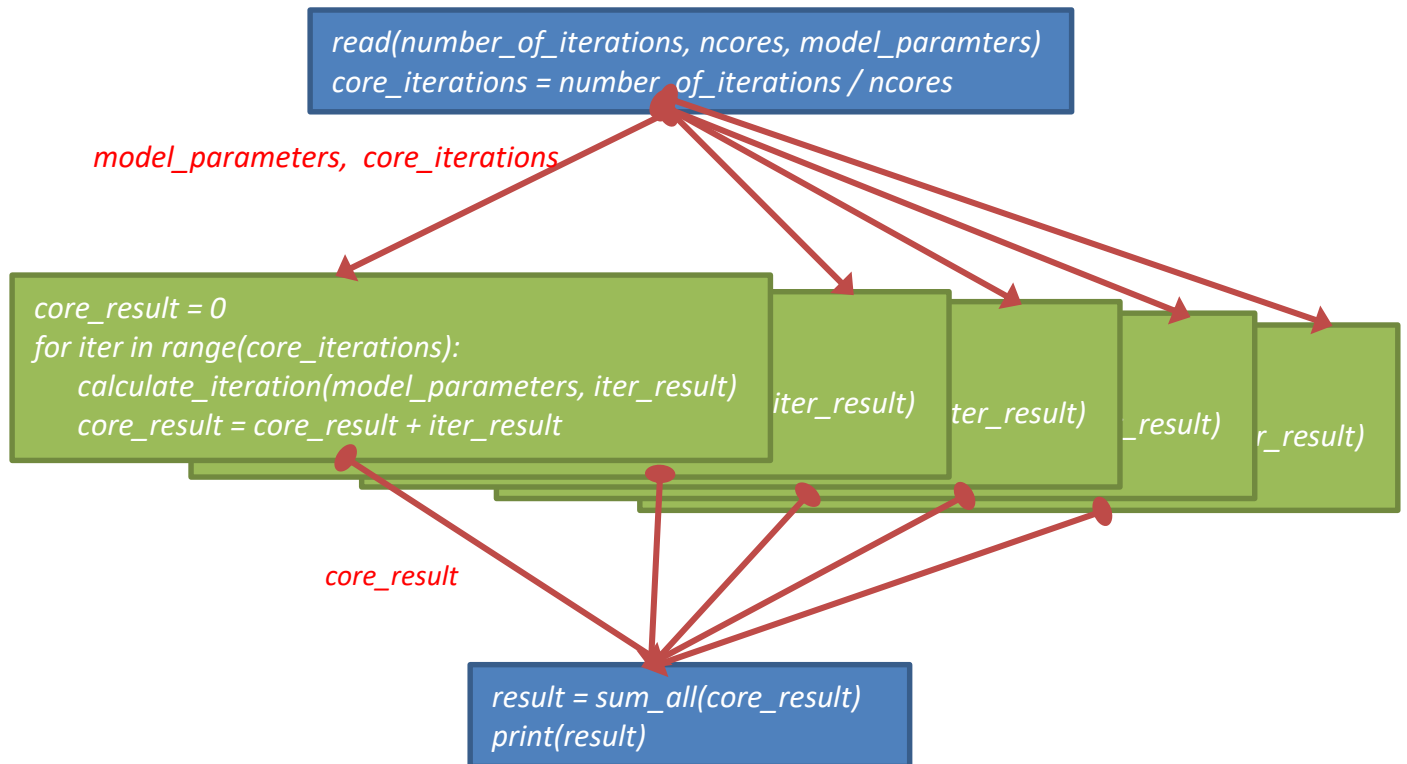
```
read(number_of_iterations, model_parameters)
result = 0
for iter in range(number_of_iterations):
    calculate_iteration(model_parameters, iter_result)
    result = result + iter_result
print(result)
```

במקרה שרוצים לחשב אינטגרל נומרי של פונקציה, ה- $model_parameters$ מגדירים את הפונקציה אותה יש לחשב, ב- $calculate_iteration$ מבוצע חישוב הפונקציה ו- $iter_result$ הוא ערך הפונקציה חלקי מספר האיטרציות הכולל. הקוד המדומה מתאים גם לתיאור חישוב של מעבר חלקיקים דרך חומר כשמתעניינים לדוגמה בכמות היחסית של החלקיקים שמגיעה לתת-אזור מסוים במערכת –אזור המטרה. במקרה זה ה- $model_parameters$ מגדיר את פרטי המערכת החומרית, החלקיקים ואזור המטרה, $calculate_iteration$ - מחשב את מעבר חלקיק אחד במערכת ו- $iter_result$ הוא אחד חלקי מספר החלקיקים הכולל אם החלקיק הגיע לאזור המטרה ואפס אחרת.

מקבול האלגוריתם במקרה זה הוא טריוויאלי. צריך לקבוע את מספר הליבות עליהם ממקבלים ($ncores$), ובכל ליבה לבצע $core_iterations = number_of_iterations / ncores$ איטרציות כאשר התוצאה נשמרת כתוצאה משנה עצמאית לכל ליבה - $core_result$. לאחר סיום החישוב בכל הליבות מסכמים את תוצאות המשנה ומדפיסים את התוצאה הכוללת.

⁴ראוי לציין שבמקרים אלו, לרוב לא ניתן כלל לבצע את החישוב באופן טורי, מכיוון שגודל הבעיה חורג ממגבלות הזיכרון של ליבה או מחשב בודד. הנוסחה במקרה זה הינה רק מדד ליעילות המקבול.

מקבול זה מתואר באיור הבא:



במצב אופייני בו מספר האיטרציות הכולל הוא גדול מאוד יחסית למספר הליבות, ושחישובי האיטרציות הם שקולים זה לזה מבחינת זמן הריצה, מקבלים יעילות מקבול גבוהה מאוד. אם יש פחות איטרציות והן לא שקולות, יתכן וזמן הריצה של אחד התהליכים המקביליים יהיה גדול יותר במידה ניכרת ממוצע האחרים ואז החישוב כולו יחכה והיעילות תיפגע (כלומר, איזון העומסים אינו טוב). ניתן לשכלל את האלגוריתם במקרה שכזה על ידי חלוקת החישוב למספר גדול יותר של תהליכים (גדול משמעותית ממספר הליבות, אך עדיין קטן בהרבה ממספר האיטרציות הכולל) ו"חלוקתם" לליבות השונות כשכל ליבה שמסיימת תהליך "מקבלת" תהליך נוסף לביצוע.

דוגמה למקבול של פתרון משוואת פואסון דו ממדית בשיטה איטרטיבית

כזכור, כדי לפתור את משוואת פואסון: $\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \rho(x, y)$ בתוך תחום במישור $[x, y]$. הגדרנו

סריג דו-ממדי במישור כך שהמיקום של נקודה $[i, j]$ יהיה בקואורדינטות $[x = ih, y = jh]$.

נסמן $u_{i,j} = u(x_i, y_j)$; $\rho_{i,j} = \rho(x_i, y_j)$

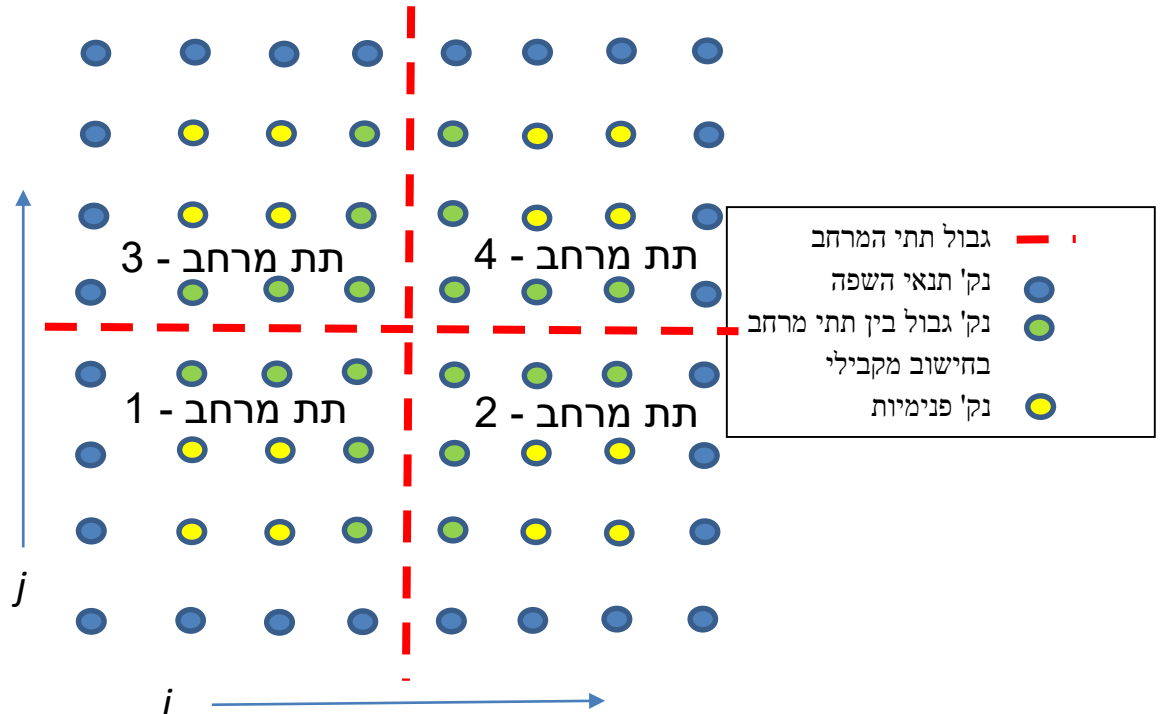
בשיטת Jacobi מחשבים את $u_{i,j}$ באיטרציות על בסיס משוואה זו. נסמן ב- $u_{i,j}^n$ את הערך של $u_{i,j}$ באיטרציה ה- n ונחשב את הערך באיטרציה ה- $n+1$:

$$u_{i,j}^{n+1} = \frac{1}{4}(u_{i+1,j}^n + u_{i-1,j}^n + u_{i,j+1}^n + u_{i,j-1}^n) - \frac{h^2}{4} \rho_{i,j}$$

ניתן לראות שהערך העדכני בנקודה מסוימת הוא ממוצע הערכים על השכנים באיטרציה הקודמת מתוקן על ידי אגף ימין של המשוואה הדיפרנציאלית.

מקבול שיטה זו אפשרי על ידי חלוקה מרחבית (domain decomposition) לפיה כל ליבת חישוב אחראית על תת מרחב כך שהמרחב כולו ממופה לכלל הליבות. כך, בכל איטרציה מחושב ערכו של u^{n+1} במקביל בכל הליבות כאשר כל ליבה מחשבת עבור ערכי $[i,j]$ המוקצים עבורה.

באיור הבא מודגמת באופן ציורי חלוקת המרחב לארבע ליבות. מכיוון שחישוב $u_{i,j}^{n+1}$ דורש את ידיעת השכנים באיטרציה הקודמת $u_{i\pm 1, j\pm 1}^n$ הרי שעבור ערכי $[i,j]$ שהם גבולות תתי המרחב (הנקודות הירוקות באיור) יש צורך במידע שחושב בליבת החישוב לה הוקצה תת המרחב השכן. לכן, נדרש בין האיטרציות שלב בו מבוצעת תקשורת בין הליבות להן מוקצים תת מרחבים שכנים. תקשורת זו מעדכנת את ערכי $u_{i,j}^n$ בנקודות הגבול. כלומר, המידע על נקודות הגבול קיים בכל הליבות הרלוונטיות, כך שכל ליבה מכירה תת מרחב המכיל גם את נקודות הגבול השכנות, ואלו מעודכנות בין איטרציות עוקבות (מקובל לקרוא לנקודות או תאי חישוב אלו תאי רפאים, ghost cells).



תוספת התקשורת בין האיטרציות פוגעת ביעילות המקבול. עם זאת, ברור מתיאור זה, שככל שהרזולוציה המרחבית משתפרת המקבול יעיל יותר מכיוון שאם נסמן ב- N את כמות הנקודות בכל ממד הרי שכמות

הנקודות בכל ליבה היא פרופורציונלית ל- N^2 וכמות נקודות הגבול היא פרופורציונלית ל- N . כלומר מקבלים שזמן הריצה המוקדש לחישוב האיטרציות עולה מהר יותר מהעלייה בכמות התקשורת הנדרשת.

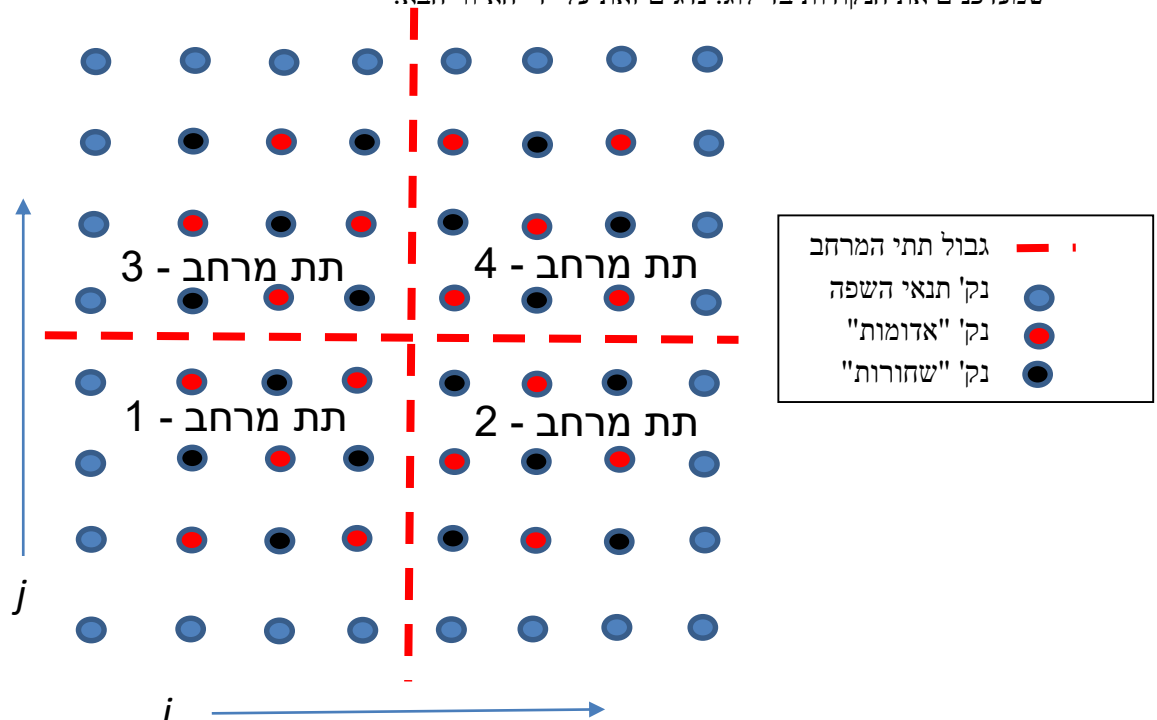
שיטת Jacobi היא שיטה איטרטיבית הדורשת איטרציות רבות. ראינו שניתן לחסוך כמחצית מהאיטרציות על ידי שיטת Gauss-Seidel בה משתמשים במידע העדכני ביותר, כלומר לפי הנוסחה:

$$u_{i,j}^{n+1} = \frac{1}{4}(u_{i+1,j}^n + u_{i-1,j}^{n+1} + u_{i,j+1}^n + u_{i,j-1}^{n+1}) - \frac{h^2}{4} \rho_{i,j}$$

ניסיון למקבל שיטה זו בשיטה זהה לשיטת Jacobi נתקל בקשיים: אם נתבונן באיור הקודם נראה לדוגמה שחישוב $u_{i,j}^{n+1}$ בנקודה הירוקה השמאלית התחתונה בתת מרחב-2, מצריך את ידיעת $u_{i,j}^{n+1}$ בנקודה הירוקה הימנית התחתונה בתת מרחב-1, כלומר לא ניתן לעדכן את ערכי $u_{i,j}^{n+1}$ במקביל בשני תתי המרחבים. תלות דומה קיימת בין נקודות הגבול בין תתי המרחב האחרים ונדרש לשנות את שיטת הפתרון.

אפשרות אחת היא לשנות את נוסחת האיטרציות עבור נקודות אלו ולהשתמש בערכי $u_{i,j}^n$ של השכנים השכנים לתתי המרחב האחרים. לפתרון זה יש שני חסרונות: הראשון הוא השיטה תתכנס לאט יותר מ-Gauss-Seidel רגיל (אך עדיין מהר יותר מ-Jacobi) והחיסרון השני הוא שנקבל שיטה נומרית שונה כאשר נגדיל את כמות הליבות כלומר נשנה את תתי המרחבים.

אפשרות אחרת היא להשתמש ב-red black ordering בה משנים את סדר המעבר על הנקודות, כך שמעדכנים את הנקודות בדילוג. נדגים זאת על ידי האיור הבא:



כך, בכל איטרציה בכל ליבה נחשב קודם את $u_{i,j}^{n+1}$ של הנקודות האדומות כשערכי השכנים (השחורים) הם מהצעד ה- n , ואח"כ את ערכי $u_{i,j}^{n+1}$ של הנקודות השחורות כשערכי השכנים (האדומים) הם מהצעד

ה- $n + 1$. בין כל שלב, מבוצעת תקשורת בין הליבות כדי לעדכן את ערכי $u_{i,j}^{n+1}$ בנקודות הגבול של תתי המרחב. ניתן לראות שזו גרסה של שיטת Gauss-Seidel מכיוון שיש רק עותק אחד של ערכי $u_{i,j}$ (בניגוד לשיטת Jacobi בה יש עותק עבור הצעד ה- n ועותק נוסף עבור הצעד ה- $n + 1$). היתרון בשיטה זו הוא שהשיטה הנומרית אינה משתנה כשמשנים את כמות הליבות למרות שגם בשיטה זו ההתכנסות פחות טובה משיטת Gauss-Seidel הרגילה.